SNOWFLAKE DATA ENGINEERING PROJECT

Healthcare Intelligence Hub

Patient, Cost & Care Analytics

Phase 5 — Project Development Phase

Institutional template-based phase document

Student	Aashi Bindal
Enrollment No.	0827CI241004
Team ID	[Team ID]
Institution	Acropolis Institute of Technology and Research · Department of CSIT
Faculty Guide	[Faculty Guide]
Academic Session	[Academic Session]
Document date	October 2026

Prepared from the verified project source (query.sql, Health/streamlit_app.py, Health/snowflake.yml, six CSV datasets). Bracketed fields are placeholders for information not supplied.

Contents

5.1 Development Environment and Setup 1

5.1.1 Repository contents (as uploaded) 1

5.1.2 Setup steps (Snowsight) 1

5.2 Verified Technology Stack and Dependencies 2

5.3 Source Data Collection and Ingestion 2

5.4 Raw-Layer Implementation 3

5.5 Data Cleaning and Standardisation (STAGING) 3

5.6 Data Transformation and Modelling (CURATED) 3

5.7 Snowflake SQL Object Inventory 4

5.8 Analytical Queries and KPIs (MART) 4

5.9 Streamlit Application Implementation 5

5.10 Integration Between Components 6

5.11 Error Handling, Logging and Validation 6

5.12 Business Questions and Visualisation Report 7

5.13 Dashboard Design 12

5.14 Story Design 13

5.15 Completed versus Planned Functionality 13

Phase 5 — Project Development Phase

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Business Question and Visualisation Report (5 marks) · Dashboard Design (5 marks) · Story Design (5 marks), from the uploaded "Tableau" template folder

> **Template adaptation.** The development templates are written for Tableau. The implemented project uses **Streamlit in Snowflake** with Altair charts, and no Tableau workbook exists in the archive. The template headings and requirements (at least 8 business questions with visualisations, dashboard outcomes, story observations, published link) are kept, and each answer points to the Streamlit or SQL component that delivers it. Where the template asks for a Tableau Public link, the field is marked *not applicable* with the Streamlit equivalent.
>
> **Figures.** Charts in this phase were regenerated from the project CSVs with the same aggregation logic as the app and views (see `tests/local_validation.py`). They are **not screenshots of the deployed Snowflake app**. Section 5.14 lists the screenshots to capture from your live app to replace or accompany them.

5.1 Development Environment and Setup

5.1.1 Repository contents (as uploaded)

```
<repo root>/                      (uploaded folder name: Nautanki)
├── query.sql                     Snowflake DDL/DML for all four layers + checks (440 lines)
├── patients.csv  doctors.csv  facilities.csv  diagnoses.csv  date.csv  patient_visits.csv
└── Health/
    ├── streamlit_app.py          Streamlit dashboard (230 lines)
    ├── snowflake.yml             Snowflake CLI project definition (v2)
    ├── pyproject.toml            Python ~=3.11, dependency streamlit[snowflake]
    └── .streamlit/config.toml    Theme placeholder (comments only)
```

No notebooks, dbt project, orchestration code, tests or README were present. The `__MACOSX/` folder in the ZIP holds macOS metadata only and is not part of the project.

5.1.2 Setup steps (Snowsight)

1. **Create objects.** Open a SQL worksheet, then run `query.sql` from the top down to and including `CREATE OR REPLACE STAGE …` (database, four schemas, six RAW tables, file format, stage).
2. **Upload files.** Upload the six CSVs to `HEALTH_DB.RAW.HEALTHCARE_STAGE`, either in Snowsight (*Data → Databases → HEALTH_DB → RAW → Stages → HEALTHCARE_STAGE → + Files*) or with SnowSQL/CLI `PUT`.
3. **Load RAW.** `query.sql` contains no load statement. Use the Snowsight *Load Data* wizard, or run the commands below. They are a **recommended addition**, not part of the original script.

```
-- Recommended load commands (not in the original query.sql)
COPY INTO HEALTH_DB.RAW.PATIENTS_RAW       FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN =
'.*patients[.]csv.*'       ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.DOCTORS_RAW        FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN =
'.*doctors[.]csv.*'        ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.FACILITIES_RAW     FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN =
'.*facilities[.]csv.*'     ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.DIAGNOSES_RAW      FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN =
'.*diagnoses[.]csv.*'      ON_ERROR = 'ABORT_STATEMENT';
```

```
COPY INTO HEALTH_DB.RAW.DATE_RAW           FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN =
'.*date[.]csv.*'           ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.PATIENT_VISITS_RAW FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN =
'.*patient_visits[.]csv.*' ON_ERROR = 'ABORT_STATEMENT';
```

The stage was created with `FILE_FORMAT = HEALTH_DB.RAW.CSV_FORMAT`, so `COPY INTO` inherits it.

4. **Transform.** Run the rest of `query.sql` (RAW count check → STAGING → CURATED → MART).
5. **Deploy the app.** Either create a Streamlit app in Snowsight (*Projects → Streamlit → + Streamlit App*, database `HEALTH_DB`, warehouse `COMPUTE_WH`) and paste `streamlit_app.py`, or from `Health/` run `snow streamlit deploy --replace` with a configured Snowflake CLI connection. The `snowflake.yml` identifier `ICYNJI7QNBXR2IUI` is an auto-generated name, which suggests the app was first created in Snowsight. Live deployment could not be verified from the files.
6. **Validate.** Run `tests/snowflake_validation.sql` (added for this documentation). Locally, `python tests/local_validation.py` validates the CSVs and the SQL logic without Snowflake.

5.2 Verified Technology Stack and Dependencies

Table 5.1 — Dependencies found in the code

Item	Version / setting	Source
Python	`requires-python = "~=3.11.0"`	`pyproject.toml`
streamlit (with `snowflake` extra)	unpinned; Snowflake supplies a pre-installed version if no repository is configured	`pyproject.toml` comments
pandas, altair	imported; installed with Streamlit	`streamlit_app.py`
snowflake-snowpark-python	`from snowflake.snowpark.context import get_active_session`	`streamlit_app.py`
uv constraint	`numba>=0.56.0` ; `exclude-newer = "7 days"`	`pyproject.toml`
Snowflake objects	warehouse `COMPUTE_WH` ; compute pool `SYSTEM_COMPUTE_POOL_CPU` (container runtime)	`snowflake.yml`

5.3 Source Data Collection and Ingestion

The six CSV files are UTF-8 with header rows, ISO `YYYY-MM-DD` dates, `True/False` booleans and `Yes/No` flags. Profiling found no blank values and no leading or trailing spaces in any column. The format below governs parsing:

```
CREATE OR REPLACE FILE FORMAT HEALTH_DB.RAW.CSV_FORMAT
    TYPE                         = CSV
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    SKIP_HEADER                  = 1
    NULL_IF                      = ('NULL', 'null', '')
    EMPTY_FIELD_AS_NULL          = TRUE
    DATE_FORMAT                  = 'YYYY-MM-DD';

CREATE OR REPLACE STAGE HEALTH_DB.RAW.HEALTHCARE_STAGE
    FILE_FORMAT = HEALTH_DB.RAW.CSV_FORMAT;
```

5.4 Raw-Layer Implementation

Six typed tables, one per source, are created with `CREATE OR REPLACE TABLE`. Re-running the script empties them, which prevents duplicates from repeated loads. Types: identifiers and text as `VARCHAR`; dates as `DATE`; costs as `NUMBER(12,2)`; satisfaction as `NUMBER(5,2)`; `IS_WEEKEND` as `BOOLEAN`; the readmission flag as `VARCHAR`. Example:

```
CREATE OR REPLACE TABLE PATIENT_VISITS_RAW (
    VISIT_ID VARCHAR, PATIENT_ID VARCHAR, DOCTOR_ID VARCHAR, FACILITY_ID VARCHAR,
    DIAGNOSIS_ID VARCHAR, VISIT_DATE DATE, VISIT_TYPE VARCHAR, ADMISSION_STATUS VARCHAR,
    TREATMENT_STATUS VARCHAR, LENGTH_OF_STAY_DAYS NUMBER,
    CONSULTATION_COST NUMBER(12,2), MEDICATION_COST NUMBER(12,2), LAB_COST NUMBER(12,2),
    ROOM_COST NUMBER(12,2), TOTAL_COST NUMBER(12,2), INSURANCE_COVERAGE NUMBER(12,2),
    PATIENT_PAYMENT NUMBER(12,2), PATIENT_SATISFACTION NUMBER(5,2),
    READMISSION_WITHIN_30_DAYS VARCHAR );   -- (formatted for print; same columns as query.sql)
```

A row-count query over all six RAW tables follows the DDL so the load can be confirmed.

5.5 Data Cleaning and Standardisation (STAGING)

Each staging table is a CTAS from its RAW table. Text is trimmed, categorical columns are case-standardised and rows without a key are removed:

```
CREATE OR REPLACE TABLE PATIENT_VISITS AS
SELECT
    TRIM(VISIT_ID)                              AS VISIT_ID,
    TRIM(PATIENT_ID)                            AS PATIENT_ID,
    ...
    INITCAP(TRIM(VISIT_TYPE))                   AS VISIT_TYPE,
    INITCAP(TRIM(ADMISSION_STATUS))             AS ADMISSION_STATUS,
    INITCAP(TRIM(TREATMENT_STATUS))             AS TREATMENT_STATUS,
    LENGTH_OF_STAY_DAYS, CONSULTATION_COST, ... , PATIENT_SATISFACTION,
    INITCAP(TRIM(READMISSION_WITHIN_30_DAYS))   AS READMISSION_WITHIN_30_DAYS
FROM HEALTH_DB.RAW.PATIENT_VISITS_RAW
WHERE VISIT_ID IS NOT NULL;
```

Three queries then report NULL keys, duplicate keys and row counts for every staging table.

Effect on this dataset (verified locally). The source is already clean, so TRIM and the NULL filter change nothing (all 14,059 rows are kept). `INITCAP` changes two values: `ENT` becomes `Ent` and `Follow-up` becomes `Follow-Up`, because Snowflake lower-cases letters after the first and treats `-` as a word separator. These become the labels shown in the app's specialty filter and visit-type charts.

5.6 Data Transformation and Modelling (CURATED)

Dimensions are `SELECT DISTINCT` copies of the staging tables. The fact table joins visit rows to descriptive attributes:

```
CREATE OR REPLACE TABLE FACT_PATIENT_VISITS AS
SELECT v.VISIT_ID, v.PATIENT_ID, v.DOCTOR_ID, v.FACILITY_ID, v.DIAGNOSIS_ID, v.VISIT_DATE,
       v.VISIT_TYPE, v.ADMISSION_STATUS, v.TREATMENT_STATUS, v.LENGTH_OF_STAY_DAYS,
       v.CONSULTATION_COST, v.MEDICATION_COST, v.LAB_COST, v.ROOM_COST, v.TOTAL_COST,
       v.INSURANCE_COVERAGE, v.PATIENT_PAYMENT, v.PATIENT_SATISFACTION, v.READMISSION_WITHIN_30_DAYS,
       p.FIRST_NAME, p.LAST_NAME, p.GENDER, p.CITY AS PATIENT_CITY, p.INSURANCE_PROVIDER,
```

```
       d.DOCTOR_NAME, d.SPECIALTY, d.DEPARTMENT,
       f.FACILITY_NAME, f.CITY AS FACILITY_CITY, f.FACILITY_TYPE,
       dg.DIAGNOSIS_NAME, dg.DIAGNOSIS_CATEGORY
FROM HEALTH_DB.STAGING.PATIENT_VISITS v
LEFT JOIN HEALTH_DB.STAGING.PATIENTS   p  ON v.PATIENT_ID   = p.PATIENT_ID
LEFT JOIN HEALTH_DB.STAGING.DOCTORS    d  ON v.DOCTOR_ID    = d.DOCTOR_ID
LEFT JOIN HEALTH_DB.STAGING.FACILITIES f  ON v.FACILITY_ID  = f.FACILITY_ID
LEFT JOIN HEALTH_DB.STAGING.DIAGNOSES  dg ON v.DIAGNOSIS_ID = dg.DIAGNOSIS_ID;
```

`LEFT JOIN` keeps visits whose reference data is missing. The orphan check that follows counts rows where the joined name is NULL; it returns 0 for this dataset (verified locally). Each key is unique in staging, so the joins do not multiply rows (FACT = 10,000).

5.7 Snowflake SQL Object Inventory

Table 5.2 — Objects created by query.sql (36 objects incl. the Streamlit app)

Schema	Object type	Count	Names
—	Database	1	`HEALTH_DB`
—	Schemas	4	`RAW` , `STAGING` , `CURATED` , `MART`
RAW	Tables	6	`PATIENTS_RAW` , `DOCTORS_RAW` , `FACILITIES_RAW` , `DIAGNOSES_RAW` , `DATE_RAW` , `PATIENT_VISITS_RAW`
RAW	File format, stage	2	`CSV_FORMAT` , `HEALTHCARE_STAGE`
STAGING	Tables	6	`PATIENTS` , `DOCTORS` , `FACILITIES` , `DIAGNOSES` , `DATE` , `PATIENT_VISITS`
CURATED	Tables	6	`DIM_PATIENT` , `DIM_DOCTOR` , `DIM_FACILITY` , `DIM_DIAGNOSIS` , `DIM_DATE` , `FACT_PATIENT_VISITS`
MART	Views	10	see Table 5.3
RAW	Streamlit	1	`ICYNJI7QNBXR2IUI` (from `snowflake.yml`)

`query.sql` also contains 19 diagnostic `SELECT` / `SHOW` statements: 2 `SHOW` commands, row counts at three layers, NULL, duplicate and orphan checks, a fact preview, and previews of all ten views.

5.8 Analytical Queries and KPIs (MART)

Table 5.3 — MART views and their measures

View	Grain	Measures
`VW_EXECUTIVE_KPI`	1 row	visits, distinct patients, doctors, facilities; total cost, insurance coverage, patient payment; avg satisfaction; avg LOS; readmissions
`VW_MONTHLY_VISITS`	month	visits, total cost, avg satisfaction
`VW_DIAGNOSIS_ANALYTICS`	diagnosis	visits, avg cost, avg LOS, avg satisfaction
`VW_FACILITY_ANALYTICS`	facility	visits, total and avg cost, avg satisfaction, avg LOS
`VW_VISIT_TYPE_DISTRIBUTION`	visit type	visits, avg cost, avg satisfaction, avg LOS (sorted by visits)
`VW_TREATMENT_ANALYTICS`	treatment status	visits, avg cost, avg satisfaction

View	Grain	Measures
`VW_ADMISSION_STATUS_ANALYTICS`	admission status	visits, avg cost
`VW_PATIENT_VISIT_TYPE_ANALYSIS`	visit type	same as `VW_VISIT_TYPE_DISTRIBUTION`, sorted by avg cost
`VW_DOCTOR_ANALYTICS`	doctor name + specialty + department	visits, avg cost, avg satisfaction, readmissions
`VW_READMISSION_ANALYTICS`	diagnosis	visits, readmissions, readmission rate %

KPI definitions:

KPI	Formula	Value (full dataset, local replica)
Total visits	`COUNT(*)`	10,000
Unique patients	`COUNT(DISTINCT PATIENT_ID)`	2,889
Total cost	`SUM(TOTAL_COST)`	82,818,797.00
Insurance coverage / patient payment	`SUM(...)`	59,918,913.84 / 22,899,883.16 (72.3% / 27.7%)
Average satisfaction	`AVG(PATIENT_SATISFACTION)`	3.83 / 5
Average LOS	`AVG(LENGTH_OF_STAY_DAYS)`	0.97 days (inpatient only: 7.69)
Readmissions / rate	`SUM(CASE WHEN flag='Yes' THEN 1 END)` / ÷ visits × 100	1,203 / 12.03%

Example view (readmission rate):

```
CREATE OR REPLACE VIEW VW_READMISSION_ANALYTICS AS
SELECT DIAGNOSIS_CATEGORY, DIAGNOSIS_NAME, COUNT(*) AS TOTAL_VISITS,
       SUM(CASE WHEN READMISSION_WITHIN_30_DAYS = 'Yes' THEN 1 ELSE 0 END) AS READMISSIONS,
       ROUND(100.0 * SUM(CASE WHEN READMISSION_WITHIN_30_DAYS = 'Yes' THEN 1 ELSE 0 END)
             / COUNT(*), 2) AS READMISSION_RATE_PCT
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1, 2
ORDER BY READMISSION_RATE_PCT DESC;
```

5.9 Streamlit Application Implementation

Data loading with caching. The app reads the fact table once and caches it for 10 minutes:

```
@st.cache_data(ttl=600)
def load_fact():
    session = get_active_session()
    df = session.table("HEALTH_DB.CURATED.FACT_PATIENT_VISITS").to_pandas()
    df.columns = df.columns.str.upper()
    df["VISIT_DATE"] = pd.to_datetime(df["VISIT_DATE"])
    return df
```

Filters. The date range is bounded by the minimum and maximum visit date. The four drop-downs are filled from the distinct values in the data, each with "All" first. Filters are applied one after another to a copy of the data frame:

```
df = df_raw.copy()
if len(date_range) == 2:
    df = df[(df["VISIT_DATE"].dt.date >= date_range[0]) & (df["VISIT_DATE"].dt.date <= date_range[1])]
if sel_facility  != "All": df = df[df["FACILITY_NAME"]      == sel_facility]
if sel_specialty != "All": df = df[df["SPECIALTY"]          == sel_specialty]
if sel_visit     != "All": df = df[df["VISIT_TYPE"]         == sel_visit]
if sel_diag_cat  != "All": df = df[df["DIAGNOSIS_CATEGORY"] == sel_diag_cat]
```

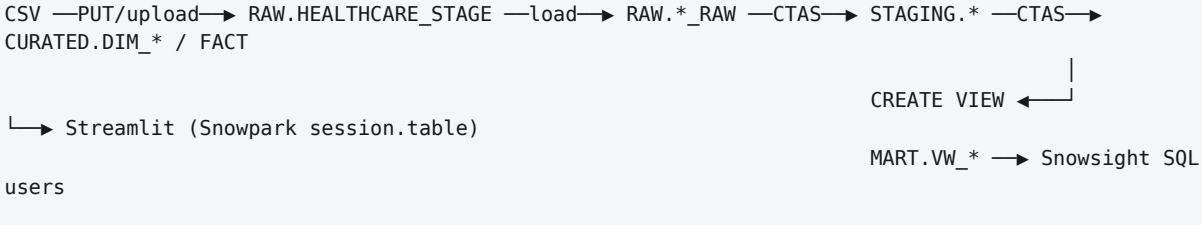
The `len(date_range) == 2` guard covers the moment when the user has picked only the first date of the range.

KPIs with empty-result guards:

```
readmit_rate = (100 * (df["READMISSION_WITHIN_30_DAYS"] == "Yes").sum() / total_visits
                if total_visits > 0 else 0)
```

Visuals: an Altair line (monthly trend, `to_period("M")`), two donuts (`mark_arc(innerRadius=50)`) for visit type and admission status, bars for top-10 diagnoses, facilities (coloured by type), treatment status and readmission rate (top 10), a stacked bar of mean cost components (`melt` to long format), and two `st.dataframe` tables (facility, doctor leaderboard). Charts include tooltips. The footer prints the filtered visit count.

5.10 Integration Between Components



The app depends only on `CURATED.FACT_PATIENT_VISITS` and its column names. Renaming a fact column breaks the app. MART views also depend on the fact table, but the app does not use them.

5.11 Error Handling, Logging and Validation

Aspect	Implemented	Gap / recommendation
Load errors	Depends on the load method (not scripted)	Use `COPY INTO … ON_ERROR='ABORT_STATEMENT'`; inspect `VALIDATE()` / `COPY_HISTORY`
Data validation	SQL checks for row counts, NULL keys, duplicate keys and orphans (manual review of results)	Make checks fail loudly, e.g., a stored procedure or test that raises when a count is > 0; add the business-rule checks from Phase 6
App robustness	Empty-filter guards for averages and rates; partial date-range guard; `dropna()` on filter options	No `try/except` around the Snowflake read; add `st.error` with a friendly message if the table is missing
Logging	None in code; Snowflake `QUERY_HISTORY` records all executed SQL	Optional: Snowflake event table for app logs

5.12 Business Questions and Visualisation Report

Field	Value
Date	[Submission Date] *(template shows 24 September 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	5 Marks

Visualisation development turns complex datasets into visual formats that are easy to interpret. The business questions below guided the selection of metrics and charts. The template requires a minimum of 8; ten are provided, including the three questions in the official problem statement. Each lists the implemented Streamlit visual and SQL view.

1. What is the monthly trend in patient visits? - Visualisation: line chart "Monthly Visit Trend" (Streamlit, Row 1); SQL: `VW_MONTHLY_VISITS`. - Finding: 32 months, 257–351 visits per month (lowest February 2026). Visits were 3,728 in 2024, 3,812 in 2025 and 2,460 in January–August 2026.

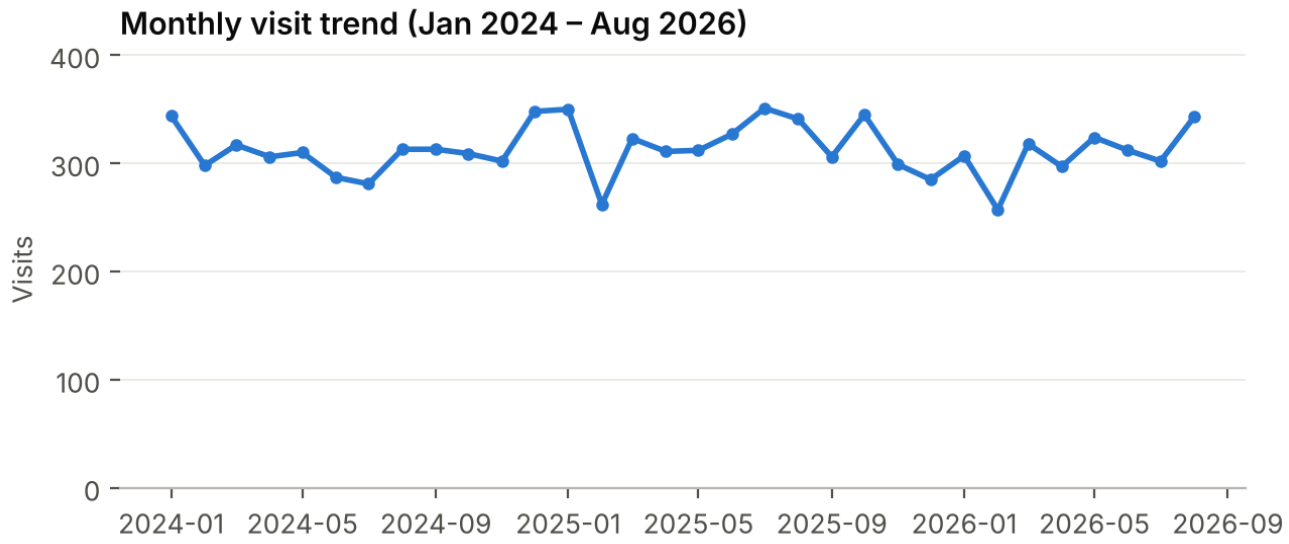


Figure 5.1 — Monthly visit trend (local reproduction from CSVs)

2. How are visits distributed across visit types? - Visualisation: donut "Visit Type Split"; SQL: `VW_VISIT_TYPE_DISTRIBUTION`. - Finding: Outpatient 39.7%, Follow-up 18.1%, Telemedicine 14.9%, Emergency 14.7%, Inpatient 12.6%.

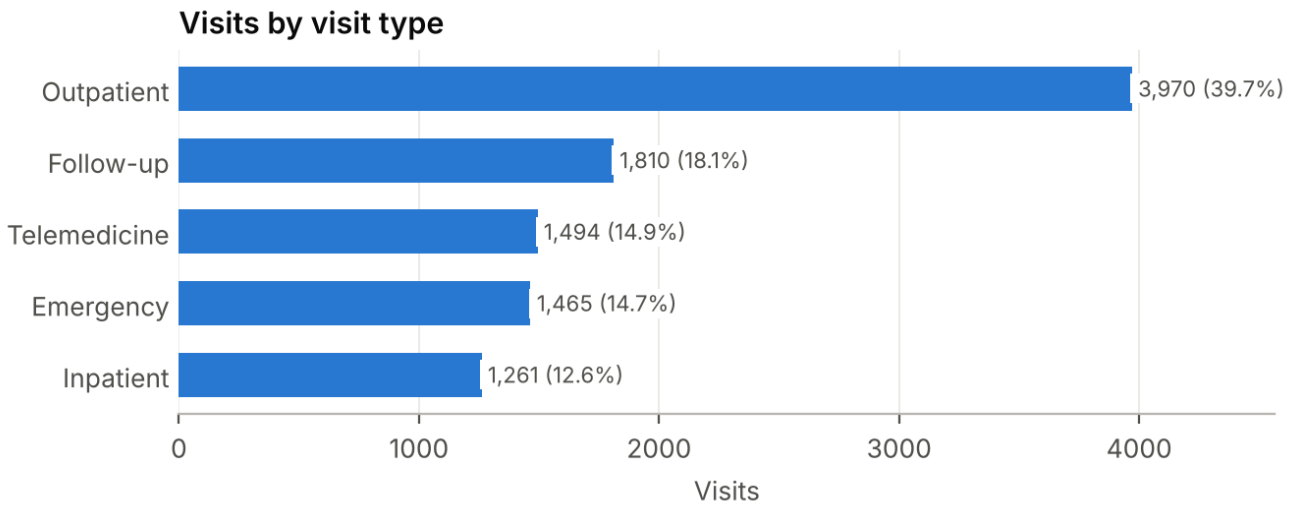


Figure 5.2 — Visits by visit type (bar form used for print; the app uses a donut)

3. Which diagnoses have the highest visit volume? - Visualisation: bar "Top 10 Diagnoses by Volume"; SQL: `VW_DIAGNOSIS_ANALYTICS`. - Finding: Pneumonia (714), Arthritis (700) and Gastritis (690) lead. Volumes are fairly even (637–714 across all 15).

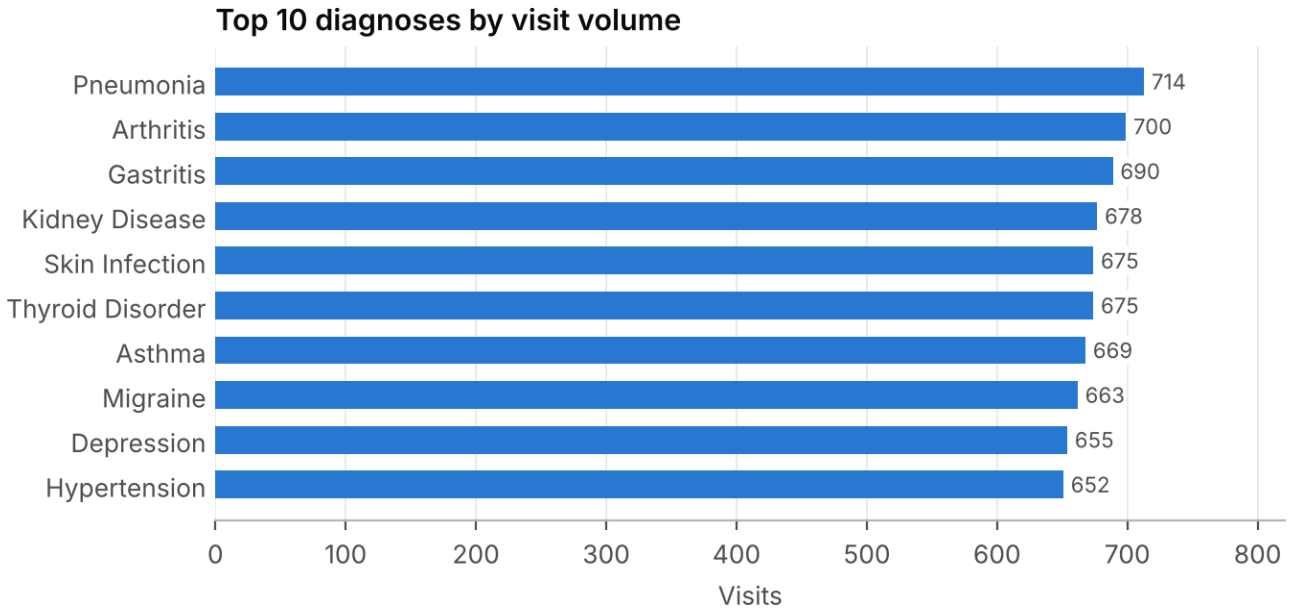


Figure 5.3 — Top 10 diagnoses by visit volume

4. Which diagnoses have the highest 30-day readmission rate? - Visualisation: bar "Readmission Rate by Diagnosis"; SQL: `VW_READMISSION_ANALYTICS`. - Finding: Depression 14.05%, Back Pain 13.98%, Asthma 13.75%, Arthritis 13.43%; Hypertension lowest at 10.43%; overall 12.03%.

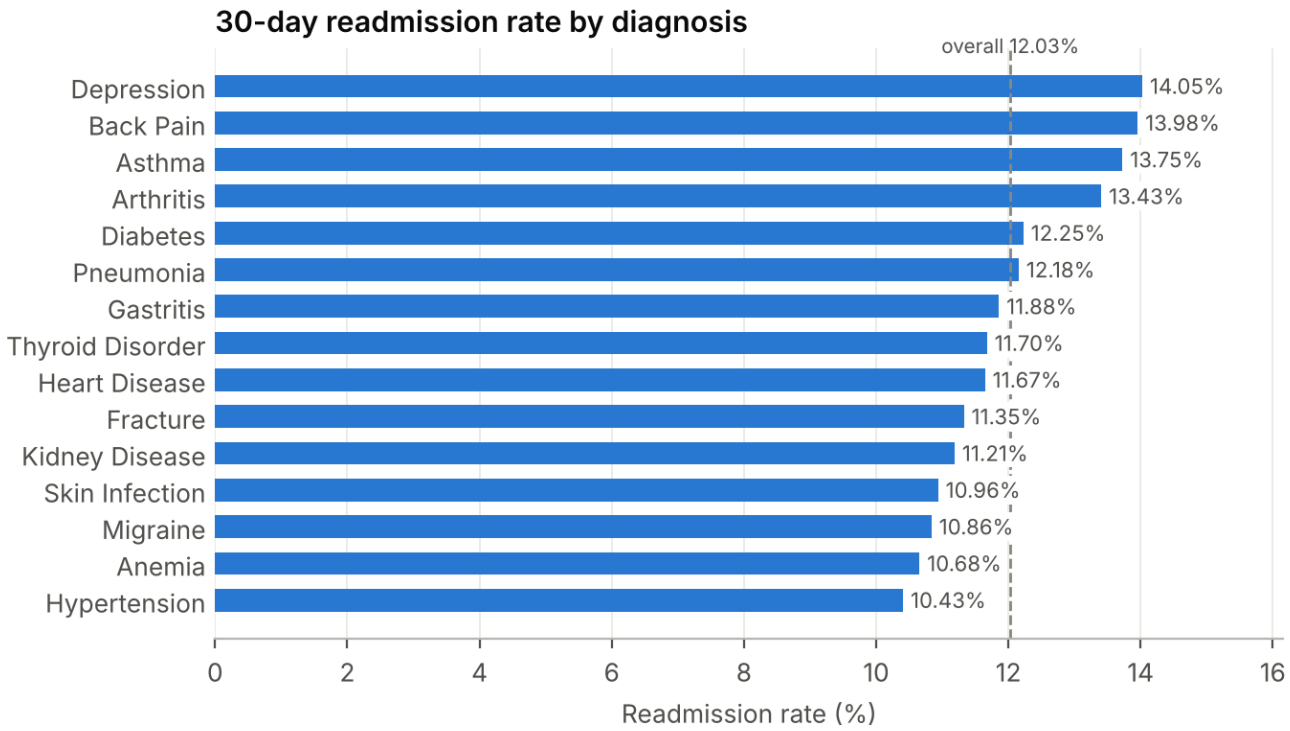


Figure 5.4 — 30-day readmission rate by diagnosis (all 15 shown; the app shows the top 10)

5. How do facilities compare on visit volume and cost per visit? - Visualisation: "Facility Performance" bar plus table; SQL: `VW_FACILITY_ANALYTICS`. - Finding: City General Hospital, Kolkata, has the most visits (1,039). Hope Hospital, Jaipur, has the highest average cost (8,432.07). Apollo View Clinic, Delhi, has the lowest (8,084.86).

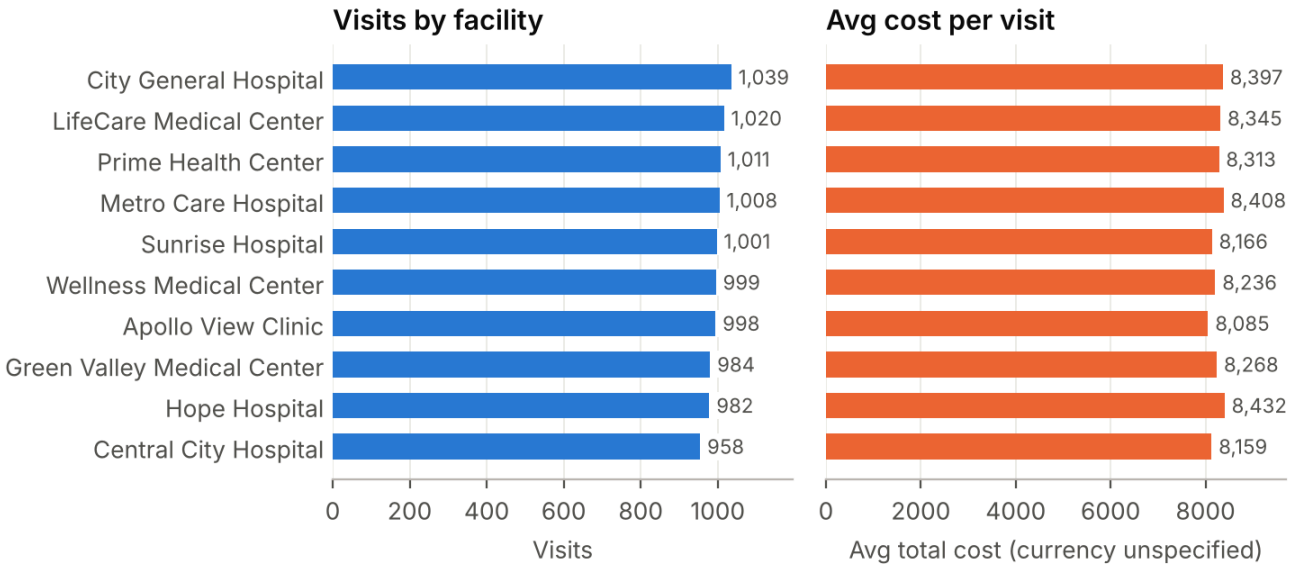


Figure 5.5 — Visits and average cost per visit by facility

6. What drives the cost of a visit for each visit type? - Visualisation: stacked bar "Average Cost Breakdown by Visit Type". - Finding: lab charges are the largest share overall (42.3% of total cost), followed by medication (30.9%). Room cost occurs only for inpatient visits (average 6,518), which nearly doubles the inpatient average to 13,973, against about 7,400–7,550 for other types.

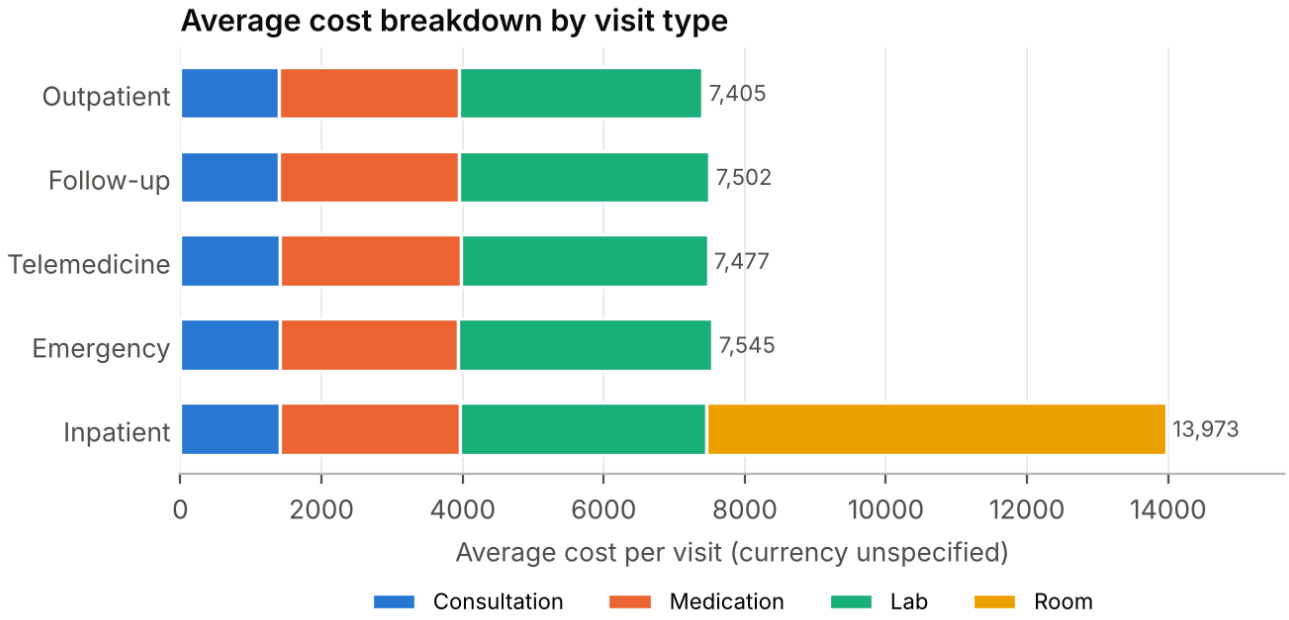


Figure 5.6 — Average cost breakdown by visit type

7. What are the treatment-outcome and admission mixes? - Visualisation: "Treatment Status" bar and "Admission Status" donut; SQL: `VW_TREATMENT_ANALYTICS`, `VW_ADMISSION_STATUS_ANALYTICS`. - Finding: 69.9% of visits are Completed, 18.8% Ongoing, 7.7% Referred and 3.6% Cancelled. 19.1% of visits are Admitted, with an average cost of 11,700 against 7,473 for Discharged.

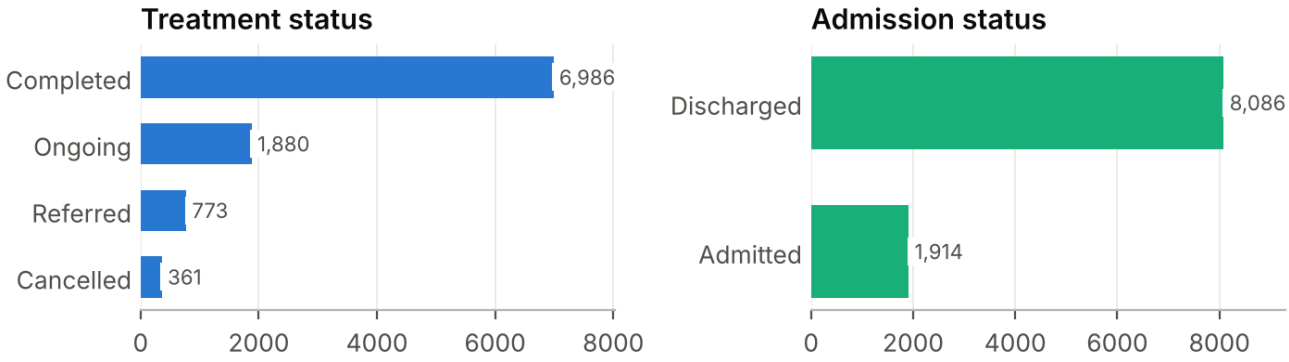


Figure 5.7 — Treatment and admission status

8. How much of the cost is covered by insurance, and does it differ by insurer? - Visualisation: SQL `VW_EXECUTIVE_KPI` (totals). There is no per-insurer chart in the app; the figure below is an additional analysis. - Finding: insurance covers 72.3% of total cost overall, and 72.0–72.7% for every provider, **including "Self Pay"**. This is a data-quality anomaly (Phase 6, TC-DQ-BR-01).

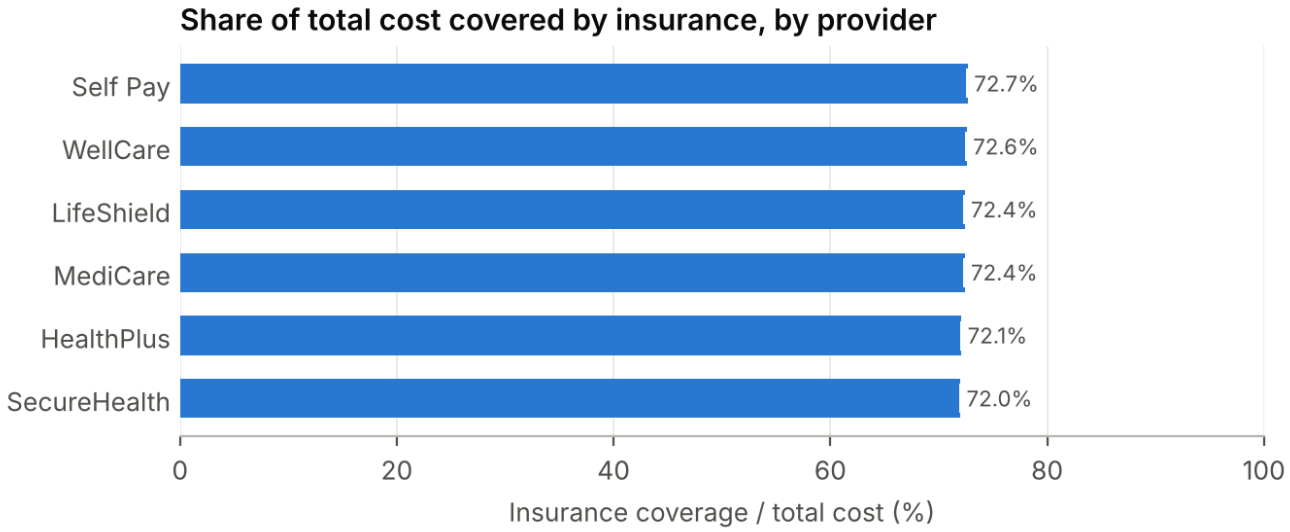


Figure 5.8 — Insurance coverage share by provider (additional analysis, not in app)

9. How satisfied are patients? - Visualisation: KPI card "Avg Satisfaction"; satisfaction columns in the facility and doctor tables. - Finding: average 3.83 / 5. 70.1% of visits score 4 or 5; 12.2% score 1 or 2. **Satisfaction at risk:** Sunrise Hospital has the lowest facility average (3.78) and the highest share of 1–2 scores (13.9%). Asthma is the lowest diagnosis (3.75). Dr. Rohan Singh (D060, 3.65) and Dr. Rohan Gupta (D004, 3.69) are the lowest doctors.

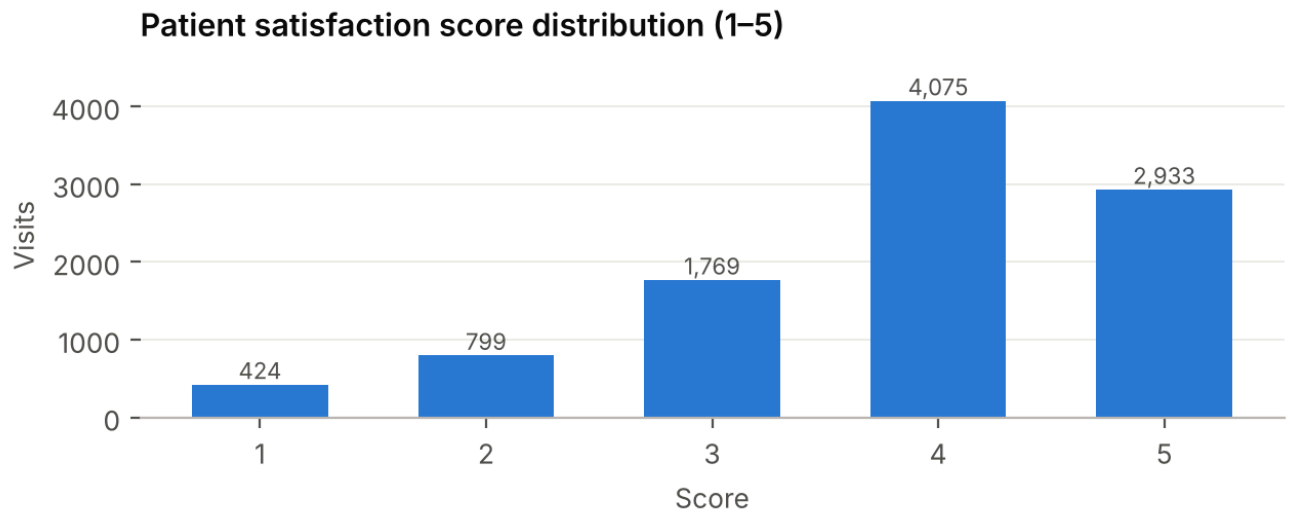


Figure 5.9 — Distribution of patient satisfaction scores (additional analysis, not in app)

10. Which diagnoses drive the longest and most expensive stays? *(question from the official problem statement)* - Visualisation: app filter Visit Type = Inpatient, then the diagnosis charts; SQL: `VW_DIAGNOSIS_ANALYTICS` (avg cost and avg LOS per diagnosis). - Finding (inpatient visits): the longest stays are Fracture (8.60 days), Anemia (8.32) and Diabetes (8.07). The most expensive are Kidney Disease (14,816.82 avg), Hypertension (14,640.75) and Asthma (14,549.67). Across all visit types, Heart Disease has the highest average LOS (1.20 days).

5.13 Dashboard Design

Field	Value
Date	[Submission Date] *(template shows 24 September 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	5 Marks

Activity 1: Interactive and visually appealing dashboards. How the implemented Streamlit dashboard addresses each design point in the template:

Template design point	Implementation in `streamlit_app.py`
Clear and Intuitive Layout	Wide layout: KPI row on top, then five rows of paired charts, then full-width cost chart and leaderboard (see Phase 3, Table 3.7)
Use Appropriate Visualisations	Line for time trend; bars for ranked comparisons; stacked bar for composition; donuts for 2–5 part shares; tables for detailed rows
Colour and Theming	Altair default categorical palette; `.streamlit/config.toml` exists but holds only comments (no custom theme)
Interactive Filters and Slicers	Date range plus four drop-downs in the sidebar
Drill-Down Capabilities	Filter-driven drill-down (e.g., choose a facility, then specialty); tooltips on every chart; sortable tables. No click-to-drill.
Responsive Design	`use_container_width=True` on all charts and tables; Streamlit columns stack on narrow screens
Custom Visuals and Icons	Not used
Use of Infographics	KPI metric cards with units

Major outcomes (full dataset, no filters):

- **Scale:** 10,000 visits from 2,889 patients across 10 facilities and 60 doctors; total cost 82.82 M (currency unspecified), of which insurance covers 59.92 M (72.3%).
- **Quality:** average satisfaction 3.83 / 5; 30-day readmission rate 12.03% (1,203 visits), highest for Depression (14.05%).
- **Utilisation:** outpatient care dominates (39.7% of visits); average LOS is 0.97 days overall but 7.69 days for inpatient stays.
- **Cost driver:** inpatient visits cost about 1.9× other visit types because of room charges. Lab is the largest cost component overall.
- **Facilities:** visit volume is evenly spread (958–1,039 per facility). Average cost per visit varies by only about 4% (8,085–8,432).

Activity 2: Published dashboard link. *Tableau Public: not applicable (no Tableau workbook in the project).* The Streamlit-in-Snowflake app is private to the Snowflake account. Its Snowsight URL can be shared only with users who have access: **[paste Snowsight app URL or a screen-recording link here]**.

5.14 Story Design

Field	Value
Date	[Submission Date] *(template shows 24 September 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	5 Marks

The template describes a Tableau Story: a sequence of visualisations that tells a narrative. No Tableau story exists in the project. The equivalent below is the narrative order used to present the Streamlit dashboard during the demonstration (Phase 8), with the screenshot to capture for each story point.

Story point	Message	Visual to capture (live app)
1. The big picture	10,000 visits, 82.8 M cost, 3.83 satisfaction, 12.03% readmissions	KPI row, no filters
2. Demand over time	Steady 257–351 visits per month	Monthly Visit Trend
3. Where care happens	Outpatient-heavy mix; facilities evenly loaded	Visit Type Split + Facility Performance
4. What it costs	Inpatient room cost doubles the visit cost	Average Cost Breakdown, filtered to Inpatient
5. Where quality slips	Depression and Back Pain readmit most	Readmission Rate by Diagnosis
6. Who delivers care	Doctor leaderboard for a chosen specialty	Doctor Performance Leaderboard filtered to one specialty

Observations:

- **Inpatient care drives cost:** inpatient visits average 13,973 against about 7,500 for other types, because only inpatient stays incur room charges.
- **Readmissions concentrate in chronic and mental-health conditions:** Depression (14.05%), Back Pain (13.98%), Asthma (13.75%) and Arthritis (13.43%) are above the 12.03% average.
- **Lab tests are the largest cost component:** 42.3% of total cost.
- **Satisfaction is uniform:** facility averages range only from 3.78 to 3.87, so satisfaction does not separate facilities in this dataset.

Activity 2: Published story link. *Tableau Public: not applicable.* **[Optional: link to a recorded demo video]**

5.15 Completed versus Planned Functionality

Table 5.4 — Implementation status (verified against code)

Feature	Status	Evidence / note
Database, schemas, RAW tables, file format, stage	Completed	`query.sql`
Load CSVs into RAW	Partially scripted	No `COPY INTO` in repo; load presumably done in the UI

Feature	Status	Evidence / note
STAGING cleaning	Completed	`query.sql` ; INITCAP side effects noted
CURATED dimensions and fact	Completed	`query.sql` ; `DIM_DATE` unused
MART KPI views (10)	Completed	`query.sql` ; doctor-view grain issue; one redundant view
Validation queries	Completed (manual review)	`query.sql`
Streamlit dashboard, 6 KPIs, 8 charts, 2 tables, 5 filters	Completed	`streamlit_app.py`
Insurance-provider analysis in app	Not implemented	Only totals in `VW_EXECUTIVE_KPI`
Weekday / weekend analysis using `DIM_DATE`	Not implemented	—
Automated refresh (Streams / Tasks)	Planned	Phase 8
RBAC and masking	Planned	Phase 3 §3.9
Automated tests	Added with this documentation	`tests/local_validation.py` , `tests/snowflake_validation.sql`